

National College of Ireland

Project Submission Sheet

Student Name: Alabi Taiwo Oluwatamilore

Student ID: 24335592

Programme: Master of Science in Artificial Intelligence **Year:** 2026/2027

Module: Intelligent Agents and Process Automation

Lecturer: Prashanth Nayak

Submission Due Date: 31st July, 2026
Customer Care Reporting Automation

Project Title:

Word Count: 2951

Academic Integrity Certificate Verification Code: bquDJveq01

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting Monday, 19th January 2026, all students are required to successfully complete the Epigeum Academic Integrity Course, which is available on Moodle. This is a self-paced online course consisting of 4 modules and will take approximately 2 hours to complete. Once you have successfully completed the course, you will receive a Certificate & Verification Code. This code must be included on the Project Cover Sheet when submitting your work. For full details and guidance, please visit: <https://libguides.ncirl.ie/academicintegrity/epigeumforstudents>

Signature: Alabi Taiwo

Date: July 30, 2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplements in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

Intelligent Agents and Process Automation

Your Name/Student Number	Course	Date
Alabi Taiwo / 24335592	Master of Science in Artificial Intelligence	July 30 th , 2026

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click [here](#).

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool
OpenAI Codex	A code-capable LLM wrapper	https://openai.com/codex/

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used.**

OpenAI Codex

To generate synthetic data; python file, gen_data.py

Prompt: I cannot use real Globacom mediation or customer data, it is confidential and GDPR protected. Write gen_data.py to make synthetic csvs that mirror the real structure, seeded so it comes out identical every time. Seed 42.

210 tickets, 5 classes x 42, built from 15 phrasings per class. Write the phrasing templates in real Glo customer voice the way I described it to you: some formal English, some Nigerian Pidgin, some all caps angry complaints. 120 customers, and deliberately break exactly 5 of them so my RPA bot has something real to catch: one bad date format, one segment that is not in the catalogue, one MSISDN prefix that is not in the catalogue, one empty first name, one MSISDN that is not 11 digits.

To be clear about what this is: I want template based generation of INPUT data, because I am not allowed to use the real data. I do not want you to invent any results. Every accuracy number, routing outcome and validation count in my report has to come from me actually running this pipeline, and it has to reproduce from the seed.

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

My student ID is: 24335592

My name is: Alabi Taiwo

The course name is: Intelligent Agents and Process Automation

Part 1: Process Identification & Description

This CA is inspired from my Huawei internship. On the Globacom telecommunications team, Fiyin, Ibrahim and Banjo, my senior colleagues lived two painful daily jobs. First, every morning someone pulled the mediation dumps, in telecom, 'mediation' is the system that collects raw usage records: (calls, data, sms data (not the actual texts), recharges) from the network and cleans them into readable files, and builds the daily customer reports, for example; "how many new customers created today, by segment?" Management needs it to know the business is alive and correct.

We used to eye check the Glo report field catalogue document; it's a rulebook spreadsheet, saying what each column is allowed to contain (MSISDN, or customer phone number must be 11 digits, segments must be of Prepaid/Postpaid/etc, date must be real).

Second, we used to read actual customer messages, and decide by hand: sim problem?, billing?, refund?, we route it to the right team, through our backend portal.

So as at the time I left Huawei, the As-is: current, manual world:

1. A human, used to be me, pulls through the dumps, I hand build the report, eyeball every field against the catalogue.

2. Another colleague, reads each customer complaint, and sorts it by hand to a team.

Both are slow, tiring, and error-prone. This is exactly what my two bots replace.

Pain 1: the manual daily report, bot: rpa_report_bot.py

The report job is rule-based and repetitive, same rules every morning (msisdn = 11 digits? valid prefix? real date?), no thinking required, just checking. My bot does what I used to do by hand.

Pain 2: the manual request sorting: AI bot triage_bot.py

Sorting complaints is not rule-based, and there's no fixed rule that maps our specific Nigerian language or lingo for example 'person comot my N2000', means; someone removed N2000 from my account, mapped to a 'billing' category. It needs language understanding: reading a sentence and judging what it means. Rules can't do that; I need Artificial Intelligence (here, a text classifier).

Also I have to add a Disclaimer: real Globacom mediation and customer data is confidential and GDPR-protected, so I did not use it. I generated small synthetic CSVs that mirror the real structure, fully reproducible from a fixed seed (42).

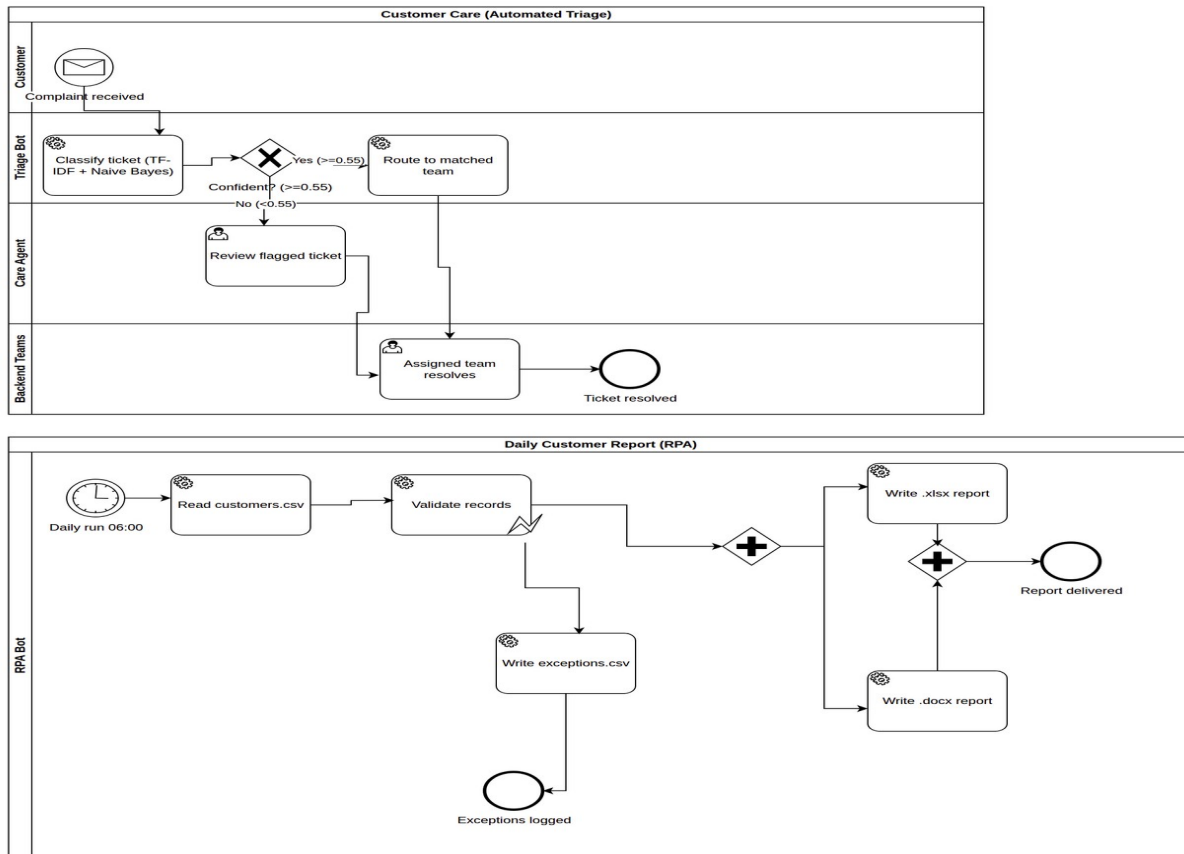
Here I will describe briefly the files in my submission:

- Config.py: it's the single source of truth; all paths, constants, as well as the 5 validation rules I mentioned earlier, plus the routing dictionary, and my defined confidence threshold.
- gen_data.py: is the seeded synthetic-data generator: 210 tickets (15 phrasings x 5 classes) + 120 customers with 5 deliberately broken records.
- triage_bot.py: My AI bot, this file, trains/fit, evaluates. Routing is programmed as; classify + route +flag.
- rpa_report_bot.py: My rpa bot; it reads, validates on my defined 5 rules, reconciles, and writes files.
- data/: Contains; tickets.csv, customers.csv, as well as new_tickets.csv.
- artifacts/: Contains the vectorizer.joblib object, as well as model_nb.joblib (the saved brain) object
- outputs/: Contains routed_tickets.csv, validation_exceptions.csv.
- diagrams/: Contains customer_care_bpmn.drawio

Part 2: Process Modelling

Both figures are present in my submission: mysubmission/diagrams/as_is.png and to_be.png

Figure 1: To-Be BPMN

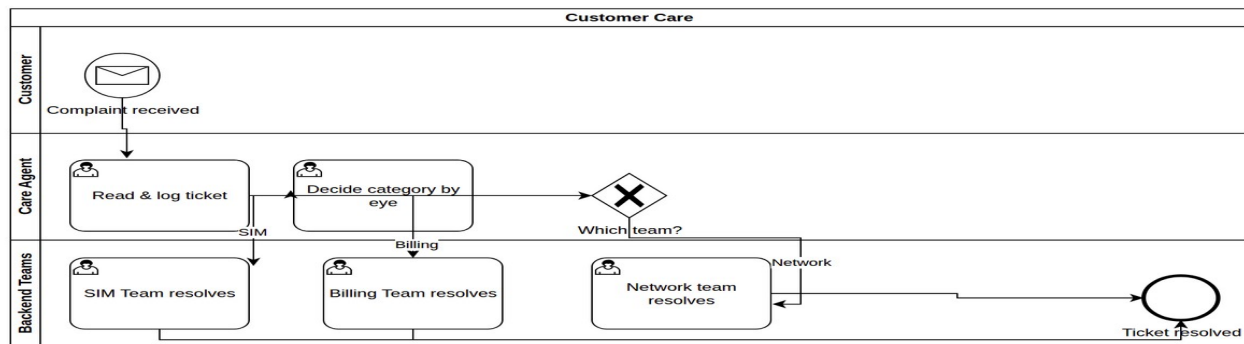


To-Be Pool 1: (Automated triage): the "decide by eye" user task becomes a service task, my Naive Bayes bot. The gateway is no longer "which team?" but "Confident? (≥ 0.55)", yes, a confidence check. If confident enough, a second service task routes automatically; if not confident enough, it will be routed to a human for review, as 'Review flagged ticket'.

To-Be Pool 2: (Robotic Process Automation): a timer start event, starts at 6:00am daily, nobody has to remember, service task reads, then validates, then I put a non-interrupting boundary error event on Validate, so bad rows leave through the side door to exceptions.csv while the good rows keep going, then a parallel (+) gateway routing to write .xlsx, as well as write .docx at once, joining, and ending at "Report delivered".

as_is.png

Figure 2: As-Is BPMN



As-Is: starts with a message start event (the customer request arrives from outside), two user tasks: one representing me, reading logs, the other, is my colleague, deciding category by eye. There is an exclusive (XOR) gateway, that directs to appropriate teams on the backend [1].

Part 3: Automation Potential Analysis

#	As-is task	Automatable	Type	Justification
1	Receive complaint	No, because it is the trigger	Human	Customers live outside the system.
2	Read & log ticket	Fully	Rpa	Same fields, same system, zero judgment
3	Decide category by eye	Fully, with a human fall back	Agentic	Needs language understanding, not rules.
4	Forward to team	Fully	Rpa	Pure rule routing.
5	Team resolves ticket	No	Human	Actual, complaints needs domain judgment.
6	Compile daily report	Fully	Rpa	The reports are scheduled, repetitive, and rule based end to end.

Rules deciding how to automate each task:

- If it follows repetitive tasks, as well as fixed rules, give it to Rpa; for example, checking if a number is 11 digits.
- If there is need for understanding of words, then give to AI, but with a confidence guard. The bot only acts when it's sure; if it's unsure, a human checks. For example; an ambiguous complaint, not sure if it should be routing to billing, or payments. Then a human in the loop is involved to make the action. For example; actually issuing the response.

Part 4: Automation Proposals: Rpa+Agentic

Solution : bot, rule-based

- The approach is a timer-triggered bot, that reads; customer csv, validate every record against the field catalogue, reconcile the counts, write the report to a .xlsx, or .docx file.
- Tools include: python (pandas + openpyxl + python-docx)
- Benefits: this method is deterministic and fast; it runs on a schedule so nobody forgets, it catches bad data instead of silently corrupting the count, the outputs are both in; (Excel for analysts, the managers prefer to read a Word document).
- Challenges: if the input schema changes; it must have the exception path, so it never drops a record without a trace. Also the field catalogue must be synched, or up-to-date with any changes in data schema

Solution : bot, agentic

- The approach is TF-IDF turns the free-text request into weighted word-numbers, the Multinomial Naive Bayes predicts the category; the bot routes on the category and flags any prediction below 0.55 for a human review
- Tools: python; scikit-learn, joblib. Multinomial NB is the fit for sparse text; it's cheap to train, and itself is a strong baseline. [3]
- Benefits: It reads free text no rule could handle, then it returns a confidence number, it keeps a human in the loop on the uncertain few instead of blindly auto-routing.
- Some of its challenges include: it needs labelled training data; it can potentially confuse categories that share vocabulary, for example both billing, as well as refunds both talk money.

Final notes:

RPA automates tasks; BPM determines what tasks to automate [4], [5], I did not start by botting something, but first I modelled the process; map first, automate second, and the model told me which tasks were rule-based (RPA) and which needed judgment by an AI. My RPA bot is just timer-triggered, no human clicks it.

Part 5: Solution Implementation: Both bots run

The run order, also present in README.md:

- python3 gen_data.py, makes the fake data first, so this builds 210 complaints, and customers.csv (120 records).
- then run; python3 triage_bot.py train; running this script teaches the AI bot, it reads the tickets, learns the patterns, and saves its 'brain', what I would call; the vectorizer + model object, to disk. This run comes before routing.
- moving on; run python3 triage_bot.py route; this script uses the trained bot, so how it works, is it loads the saved brain and sorts new complaints into categories, as well as flags the doubtful ones. By the way, the script only works after step 2 saves the brain.
- then run; python3 rpa_report_bot.py, this runs the rpa bot; reads customers.csv, validates every record, builds the daily report (in both .xlsx, as well as .docx), and the exceptions file.

How myAI bot works:

TF-IDF turns text into numbers: a computer can't do maths on words, so this converts each complaint into numbers, giving rare useful words (like "refund", "prefix") more weight than common ones (like "the", "my"), like a word-importance scale.

Multinomial Naive Bayes predicts the category: NB is the actual classifier. It looks at those word-numbers and picks the most likely category.

joblib saves/reloads the artifacts: after training, the 'brain'; the learned word-scale + the model, gets saved to disk in a jar, so route can open the same jar later instead of relearning. Artifacts = the saved brain files. The word-scale; vectorizer, learns its vocabulary from the training data only, then gets saved and reused on the test data. So the model never sees the test words while learning.

I decided on a stratified 80/20 split: I split the 210 tickets: 80% to learn from, 20% to test on.

Stratified, so each category keeps its fair share in both halves, not all refunds landing in one pile. I set random_state to 42, a fixed seed so the split is identical every run.

Confidence: for each prediction, the bot gives a probability for its top pick for example, 'i'm 0.68 sure this is billing'.

CONF_THRESHOLD: I set to 0.55. If confidence falls below 0.55, the ticket gets flagged to needs_human_review for a human to review.

How the bot works:

There are five validation rules, each record must pass:

1. Required fields present; there should be no important box left empty; no missing name, ID, etc.
2. MSISDN should be 11 digits, and start with Glo prefix; for example, 0805, 0705, not 0999.
3. Known segment; the customer type must be one the catalogue knows:
Prepaid/Postpaid/Corporate/SME
4. Known status; the account must be a real one: Active/Pending/Suspended
5. Real date; the creation date must be an actual YYYY-MM-DD date, not a date like '07/07/2026'

The flow; clean rows → report; broken rows, write to → validation_exceptions.csv. Records that pass all five go into the daily report. Records that fail any rule get set aside into a separate exceptions file, each with a note saying which rule it broke.

Part 6: Results, Counted Properly

Tickets: There are 15 skeleton phrasings, for the 5 classes, there are 42 rows per class and a total of 210 tickets.

I will explain the data math first; I wrote 15 different ways to phrase a complaint for each of the 5 categories. So then the generator loops 42 times, and each loop it randomly grabs one of the 15 phrasings and fills the blanks. It does not use each phrasing once, it picks with repeats. Some of the 15 get picked 2 or 3 times, some maybe 4, it's random, but 42 draws total.

It's important I explain why they're not duplicates even when the same phrasing repeats; the blanks get filled differently each time. For example this SIM phrasing:

- *'Please activate my line {msisdn}, I bought it in {location}'*

If the loop picks it twice and I get two different tickets:

- *'Please activate my line 08051234567, I bought it in Lekki'*

- *'Please activate my line 07059988776, I bought it in Ibadan'*

This way the bot sees varied text.

Train/Test Split:

I split the 210 tickets like this; 80% (168) to learn from, 20% (42) to test on. I chose a stratified split for this, so each category keeps its fair share in both halves. I set the random state to 42. A fixed seed so the exact same split happens every run, and it's reproducible when you run it.

Vocabulary:

When TF-IDF learned from the 168 training tickets, it found 358 unique words worth tracking. There are 42 test tickets (mixed categories), not all under one class. So each row is scanned against all 358 words. For example;

		<i>word1</i>	<i>word2</i>	<i>word3</i>	...	<i>word358</i>	
<i>ticket 1</i>	[	0.0	0.7	0.0	...	0.0	]
<i>ticket 2</i>	[	0.4	0.0	0.0	...	0.2	]
...							
<i>ticket 42</i>	[	0.0	0.0	0.9	...	0.0	]

Most cells are 0, because any one ticket only uses a handful of the 358 words.

Edge Case Scenarios:

From all 120 customer rows, I intentionally broke 5, each in different ways, so the Rpa validator has something real to catch:

1. An unknown segment 'VIP': I stated this earlier, this isn't in the catalogue. (only Prepaid, Postpaid, Corporate, SME allowed)
2. bad date format: '07/07/2026' instead of proper YYYY-MM-DD
3. bad prefix 0999: Also not a real Glo prefix.
4. empty FirstName: A required field left blank.
5. too short MSISDN: fewer than 11 digits.

AI bot, triage_bot.py:

There are 4 measures: per category

- precision: when the bot says '*this is X*', how often is it right? For example, of everything it called refund, how much really was refund?
- recall: of all the real X's, how many did the bot catch? For example, of all the actual refunds, how many did it find
- f1: the fair average of precision and recall, it punishes if either are weak.
- support: how many real examples of that category were in the test pile. In my test set, across all categories; $9+8+8+9+8 = 42$.

What the numbers are actually saying:

- Three categories are perfect; 1.00: they are: General, Network, Sim. Their words are distinctive, for example: '*no signal*,' '*activate my SIM*'
- The only two that slip are Payment_Billing (0.88) and Refunds (0.89) category. Those are my 2 wrong tickets.

error-analysis:

The bot's only confusion is billing, and refunds. This is why:

- billing: '*I was charged N2000 for data I never got*' is very similar to,
- refund: '*Refund my N2000, I was charged for data I never got*'

Same money-words. '*charged*,' '*deducted*,' '*N2000*' appear in both. The bot mixes them, most likely where a tired human would probably too, these two genuinely blur in real life. So it's not a bug, it's the model behaving realistically.

Honest limitation:

it's a small synthetic set, so the test pile is only 42.

Figure 3 - AI bot classification report per category. Three categories score a perfect 1.00 across precision, recall and F1: SIM, Network and General. Their complaint words are distinctive enough that the bot never mislabels them. The two that dip are Payment/Billing at 0.88 and Refunds at 0.89 - those 2 wrong tickets come from the money-word overlap I describe below.

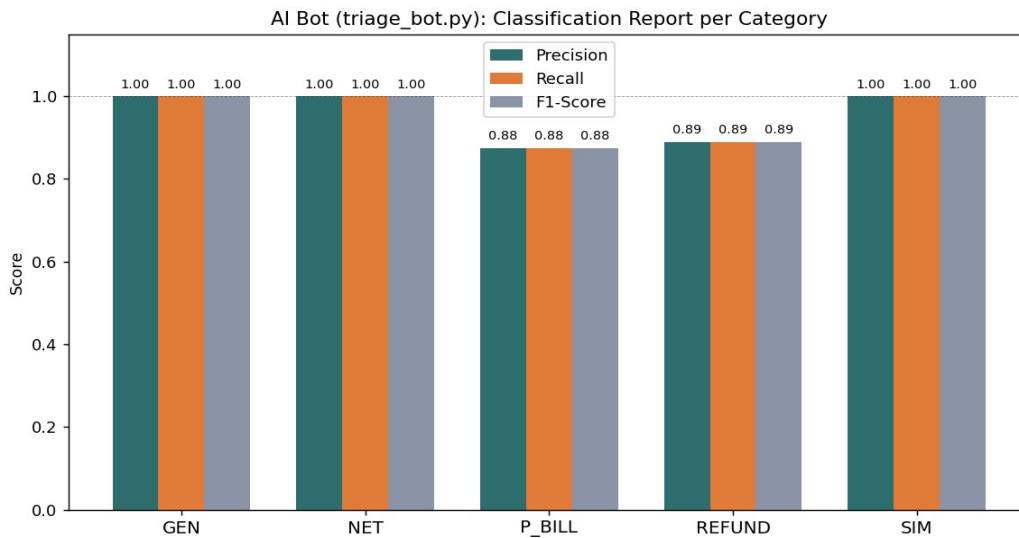


Figure 4 - confusion matrix. Each row is the actual category, each column is what the bot predicted. The only off-diagonal values are between Payment,Billing and Refunds: 1 refund ticket got called billing, and 1 billing ticket got called refund. Every other category has zero mistakes. This is the billing-refund blur the error analysis section describes.

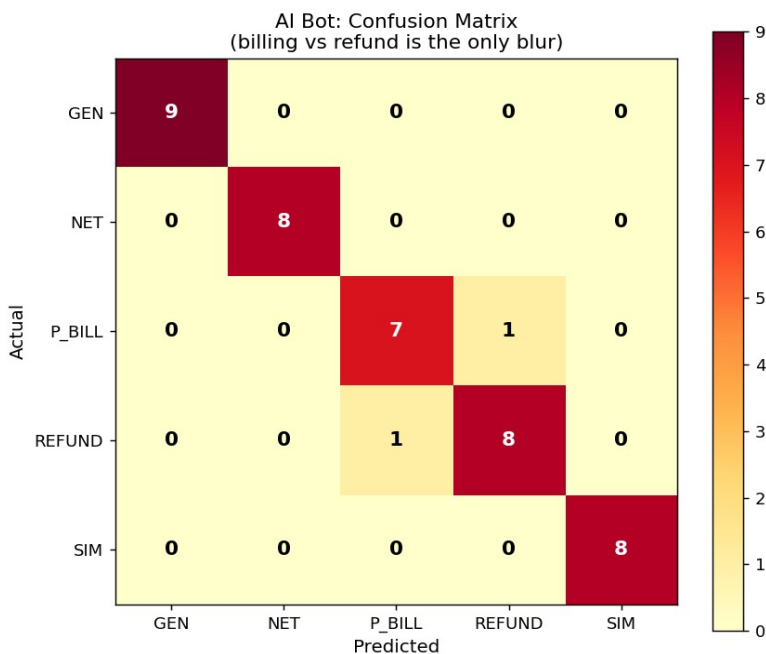


Table:

Category	precision	recall	f1	support
GENERAL	1.00	1.00	1.00	9
NETWORK	1.00	1.00	1.00	8
PAYMENT_BILLING	0.88	0.88	0.88	8
NG				

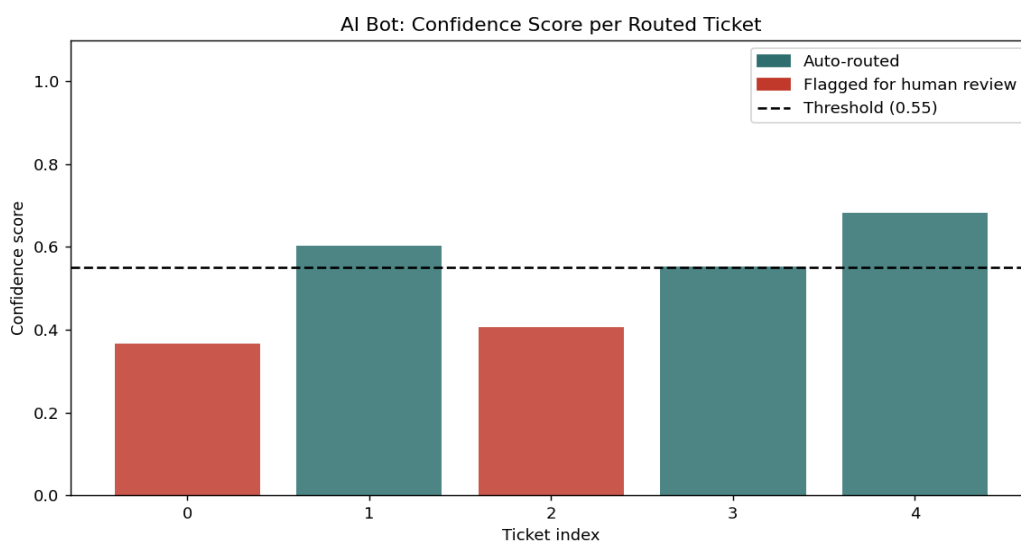
Category	precision	recall	f1	support
REFUNDS	0.89	0.89	0.89	9
SIM_ACTIVATIO	1.00	1.00	1.00	8
N				
accuracy			0.952	42

AI bot, triage_bot.py: live demo:

This is the bot doing its real job on 5 brand-new tickets it's never seen. The two things it reports are:

- 5 out of 5 got the correct category; every ticket was labelled right. The bot's judgement is perfect.
- But 2/5 were flagged for a human anyway; *'if all 5 were right, why flag 2?'*, because being right and being sure are different things. The bot doesn't know it was right, it only knows how confident it feels. The SIM and payment tickets were correct, but a bit shaky, the bot's top guess only scored 0.37 and 0.41, below my 0.55 line. So the bot says; *'I think it's SIM, but I'm not sure enough. Let a human check'*. The confident three (network 0.60, refund 0.55, general 0.68) auto-routed. This is the confidence gate working.

Figure 5 - confidence score per routed ticket. The dotted line at 0.55 is my CONF_THRESHOLD. Tickets above the line get auto-routed; tickets below get flagged needs_human_review. On the 5 live demo tickets: 5 out of 5 got the correct label, but 2 fell below the threshold and were still flagged. Being right and being confident are two different things - the human-in-the-loop guard is working exactly as designed.



RPA bot, rpa_report_bot.py:

Out of 120 records, 115 passed, all five rules, 5 failed. All five broken records got caught by the correct rule.

The 5 go to validation_exceptions.csv with their reasons, so a human sees what was wrong. Nothing dropped silently.

Report day counts;

The day report shows SME 5, Postpaid 3, Prepaid 2, Corporate 1 = 11 customers. Not 115, because the daily report isn't, all clean customers, ever, instead it's new customers created on ONE specific day. The bot:

1. keeps the 115 clean records, then
2. filters to just the report date 2026-07-07, for example, which had 11 new customers.

Figure 6 - RPA bot validation results across all 120 customer records. 115 records passed all five catalogue rules and went straight into the daily report. 5 records failed and were written to validation_exceptions.csv with their exact reasons attached, so a human can see what broke and fix it. Nothing failed silently. The right panel shows which specific rule caught each broken record.



The outputs:

- .xlsx; 3 sheets: contains the 11 detail rows, the raw truth, if they want to inspect a specific customer. + summary-by-segment: SME 5, Postpaid 3, Prepaid 2, Corporate 1; the breakdown, as well as its summary-by-status.
- .docx; a clean title, as well a table, I design it this way, because, then the manager just glances.

Both in mysubmission/outputs/.

Part 7: References

- [1] OMG, "Business Process Model and Notation (BPMN), Version 2.0", Object Management Group, 2011.
- [2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [3] A. McCallum and K. Nigam, "A Comparison of Event Models for Naive Bayes Text Classification", AAAI-98 Workshop on Learning for Text Categorization, 1998.
- [4] M. Dumas, M. La Rosa, J. Mendling and H. A. Reijers, Fundamentals of Business Process Management, 2nd ed., Springer, 2018.
- [5] W. M. P. van der Aalst, M. Bichler and A. Heinzl, "Robotic Process Automation", Business & Information Systems Engineering, vol. 60, no. 4, pp. 269-272, 2018.